

Prédiction du Diabète — Pipeline MLOps de bout en bout

Module : MLOps et Déploiement de modèles

Institut Supérieur du Numérique — SupNum · Master DEML (M1) — 2026

Enseignant : Yehdih ANNA · Date : 3 juillet 2026

Groupe 1 : Bechir Mady (22014) · Abdarahman Abdarahman (22053) · Aly Wedad (22086)

1. Liens d'accès

- **Repository GitHub** : <https://github.com/AymanMady/Projet-MLOps>
- **Serveur MLflow (Tracking + Registry, EC2 #1)** : <http://54.95.120.104:5000>
- **Application Flask (EC2 #2)** : <http://54.95.120.104:5001>
- **Stockage S3** : bucket m1-2026-mlops-project-group1 (données brutes / prétraitées, cache DVC, artefacts).

2. Présentation du problème ML et du dataset

Problème traité

Il s'agit d'un problème de **classification binaire supervisée** en santé : prédire si un patient est *diabétique* (`Outcome` = 1) ou *non diabétique* (`Outcome` = 0) à partir de mesures cliniques simples. L'objectif applicatif est un outil d'aide au dépistage précoce, exposé via une interface web où un utilisateur saisit les mesures d'un patient et obtient une prédiction immédiate.

Jeu de données

Nous utilisons le dataset **Pima Indians Diabetes** (~768 observations, 8 variables explicatives numériques + 1 cible). Les données brutes sont versionnées avec DVC et stockées sur S3.

Variable	Description	Type
Pregnancies	Nombre de grossesses	entier
Glucose	Glycémie (test de tolérance)	numérique
BloodPressure	Pression artérielle diastolique (mm Hg)	numérique
SkinThickness	Épaisseur du pli cutané tricipital (mm)	numérique
Insulin	Insuline sérique (mu U/ml)	numérique
BMI	Indice de masse corporelle	numérique
DiabetesPedigreeFunction	Antécédents familiaux (score)	numérique
Age	Âge (années)	entier
Outcome	Cible : 1 = diabétique, 0 = non	binaire

TABLE 1 – Variables du dataset Pima Indians Diabetes.

Modèle & métriques. Nous entraînons un `RandomForestClassifier` (scikit-learn ; `n_estimators=200`, `max_depth=8`). Les données sont nettoyées (suppression des doublons, imputation médiane) puis mises à l'échelle (`StandardScaler`). Les performances sont suivies via l'**accuracy**, le **F1-score**, la **précision** et le **rappel**, tous logués dans MLflow.

3. Architecture technique mise en place

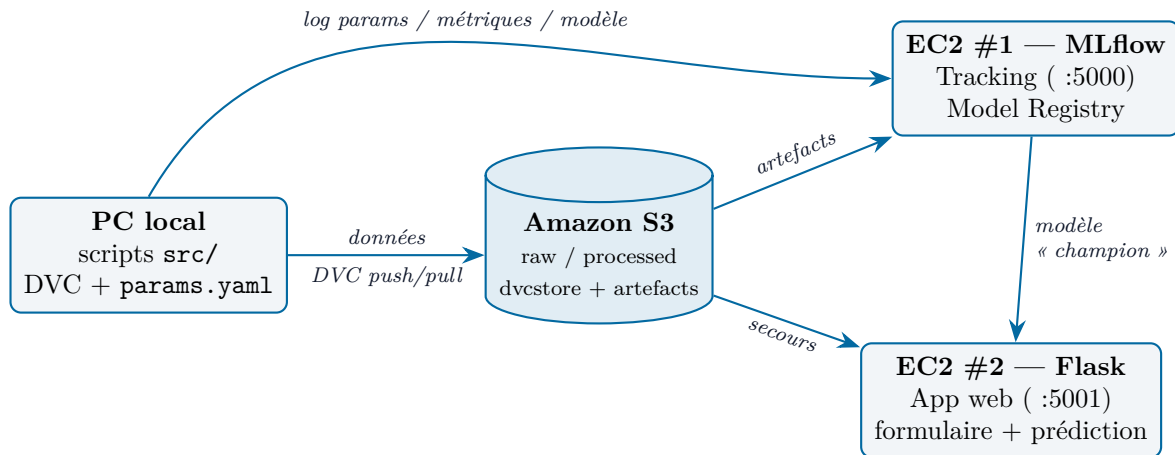


FIGURE 1 – Architecture : le poste local orchestre le pipeline, S3 centralise les données/artefacts, EC2#1 héberge MLflow (tracking + registry) et EC2#2 sert le modèle via Flask.

Rôle de chaque composant :

- **PC local** — exécute les scripts (`preprocess` → `train` → `evaluate`), pilote DVC et logue vers MLflow.
- **Amazon S3** (m1-2026-mlops-project-group1) — dépôt partagé : données brutes, données pré-traitées, *remote* DVC (`dvcstore`) et artefacts/modèles MLflow.
- **EC2 #1 — MLflow** — serveur de *tracking* des expériences et *Model Registry* (gestion des versions + alias « champion »).
- **EC2 #2 — Flask** — application web qui charge le modèle « champion » du Model Registry et renvoie la prédiction ; repli sur le modèle local si le Registry est indisponible.

4. Pipeline ML et suivi des expériences (MLflow)

Trois scripts, intégrés à MLflow dès le départ :

1. `src/preprocess.py` — nettoyage, imputation médiane, `StandardScaler` ; écrit les données pré-traitées sur S3.
2. `src/train.py` — entraîne le `RandomForest`, logue les **hyperparamètres** et les **métriques** (accuracy, F1), et enregistre le modèle comme artefact.
3. `src/evaluate.py` — évalue (accuracy, F1, précision, rappel) et **n'enregistre que le meilleur modèle** dans le Registry (si $\text{accuracy} \geq \text{seuil}$ et $\geq$ meilleure version existante), puis lui affecte l'alias « champion ».

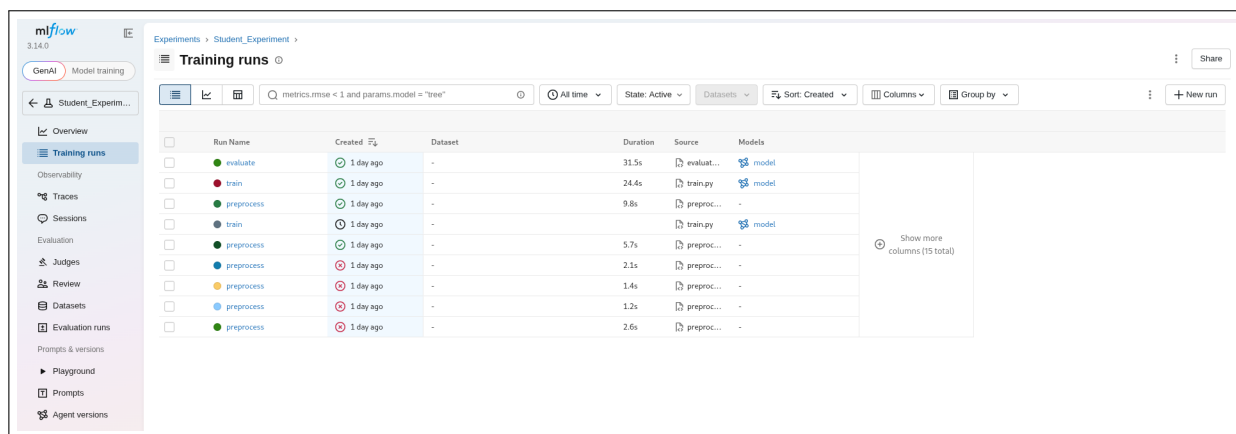


Figure 2 shows the MLflow 'Training runs' interface for the 'Student_Experiment'. The table lists various runs, including 'evaluate', 'train', and 'preprocess' steps, with their respective durations and statuses.

Run Name	Created	Dataset	Duration	Source	Models
evaluate	1 day ago	-	31.5s	evaluat...	model
train	1 day ago	-	24.4s	train.py	model
preprocess	1 day ago	-	9.8s	preproc...	-
train	1 day ago	-	-	train.py	model
preprocess	1 day ago	-	5.7s	preproc...	-
preprocess	1 day ago	-	2.1s	preproc...	-
preprocess	1 day ago	-	1.4s	preproc...	-
preprocess	1 day ago	-	1.2s	preproc...	-
preprocess	1 day ago	-	2.6s	preproc...	-

FIGURE 2 – MLflow — liste des runs de l'expérience « Student_Experiment » (preprocess, train, evaluate).

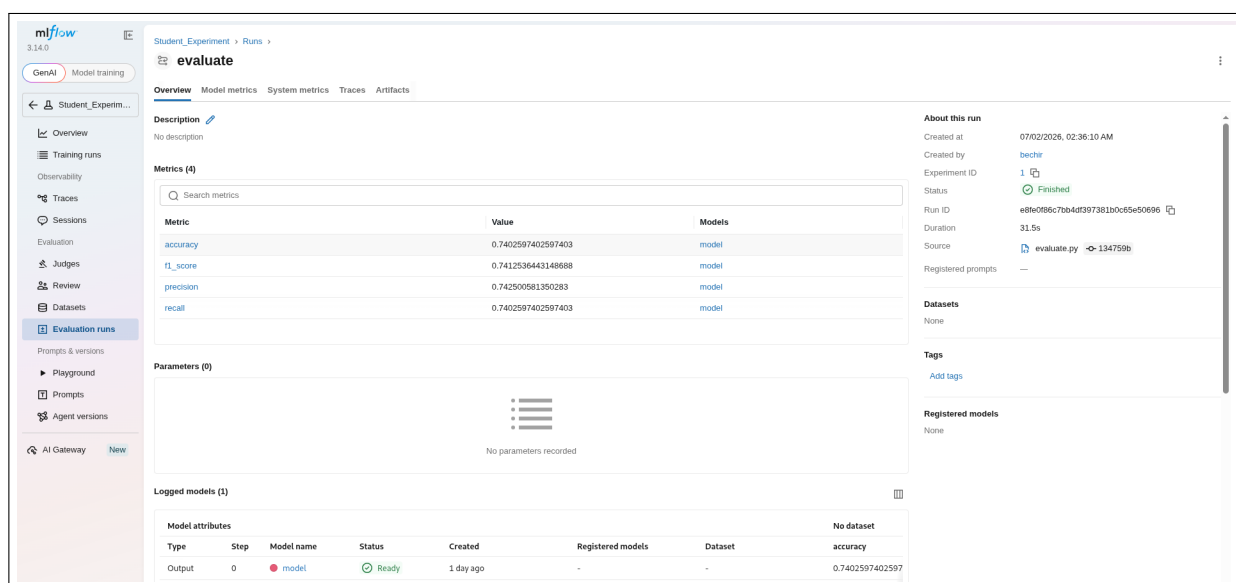


Figure 3 shows the details of an 'evaluate' run. It includes a table of metrics (accuracy, f1_score, precision, recall) and a table of logged models.

Metric	Value	Models
accuracy	0.7402597402597403	model
f1_score	0.7412536443148688	model
precision	0.742500581350283	model
recall	0.7402597402597403	model

Type	Step	Model name	Status	Created	Registered models	Dataset	No dataset
Output	0	model	Ready	1 day ago	-	-	accuracy 0.7402597402597

FIGURE 3 – MLflow — détail d'un run : hyperparamètres (n_estimators, max_depth...) et métriques (accuracy, F1, précision, rappel).

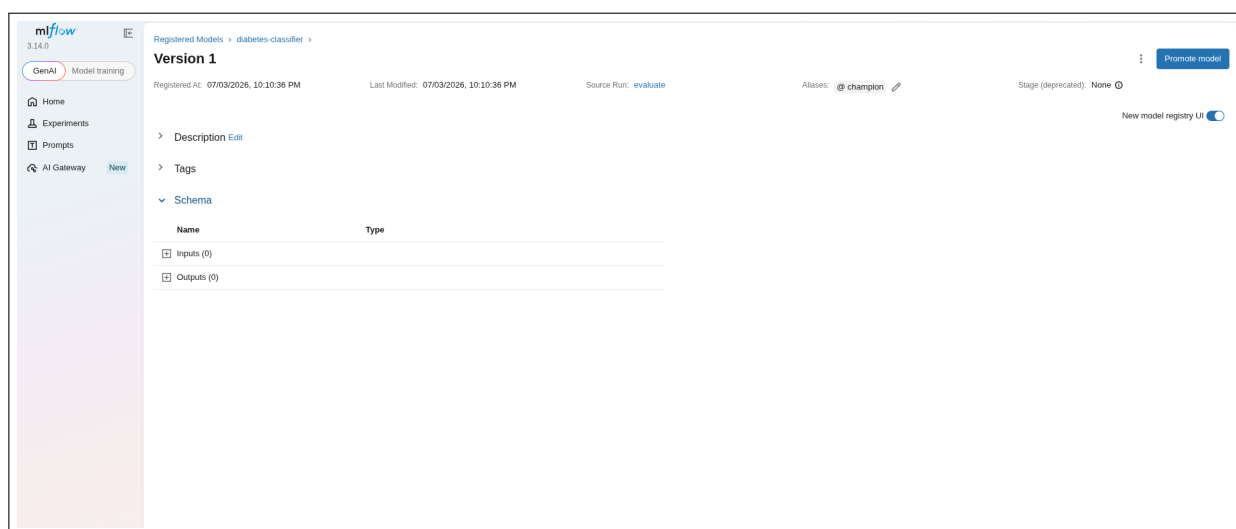


Figure 4 shows the MLflow Model Registry for the 'diabetes-classifier' model. It displays the model's version (Version 1), its description, tags, and schema.

Name	Type
Inputs (0)	
Outputs (0)	

FIGURE 4 – MLflow — Model Registry : modèle « diabetes-classifier », ses versions et l'alias « champion ».

5. Données & reproductibilité (DVC + S3)

Les données sont versionnées avec **DVC**, en utilisant S3 comme *remote storage* (dossier `dvcstore` du bucket).

```
(base) bechir@fedora:~/Documents/S2/MLOPS/projet/Projet-MLOps$ dvc push
Collecting
Pushing
1 file pushed
(base) bechir@fedora:~/Documents/S2/MLOPS/projet/Projet-MLOps$ dvc status
Data and pipelines are up to date.
```

FIGURE 5 — DVC — terminal montrant `dvc status` / `dvc push` (ou `dvc dag`) et le versioning des données.

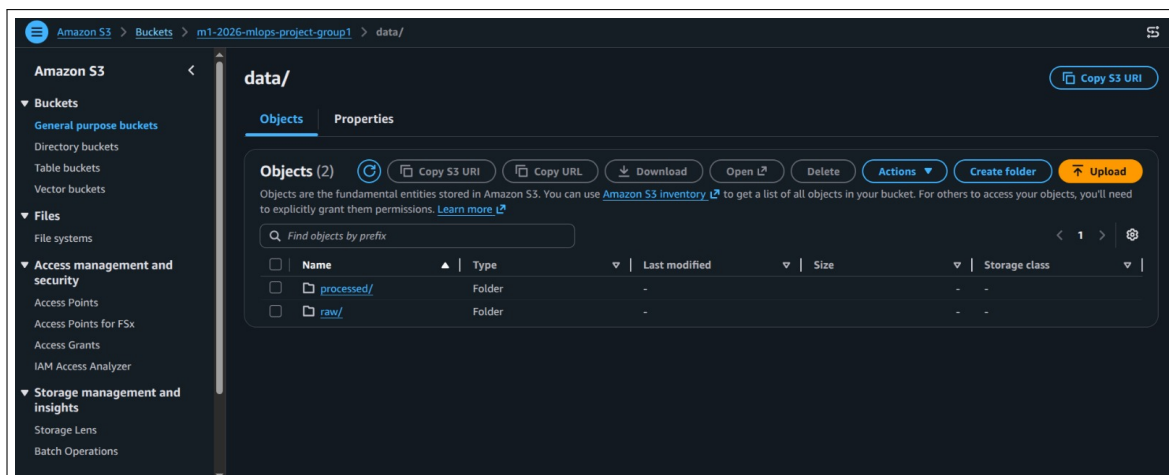


FIGURE 6 — Console AWS S3 — bucket « m1-2026-mlops-project-group1 » avec les dossiers `data/raw`, `data/processed` et `dvcstore`.

6. Déploiement — Application Flask

L'application `app/app.py` expose un formulaire (les 8 features cliniques). À la soumission, elle charge le modèle « champion » depuis le Model Registry MLflow, effectue la prédiction et affiche le résultat (« Diabétique » / « Non diabétique »). Elle est déployée sur une seconde instance EC2 (port 5001).

FIGURE 7 — Interface Flask — formulaire de saisie des mesures.

FIGURE 8 — Interface Flask — résultat de la prédiction affiché.

7. Collaboration & historique Git

Le dépôt suit un modèle multi-branches : chaque membre dispose d'une branche personnelle (bechir-22014, abdrahman, aliy-22086) fusionnée dans `main` via des *Pull Requests*, produisant un arbre Git non linéaire avec plusieurs *merges*.

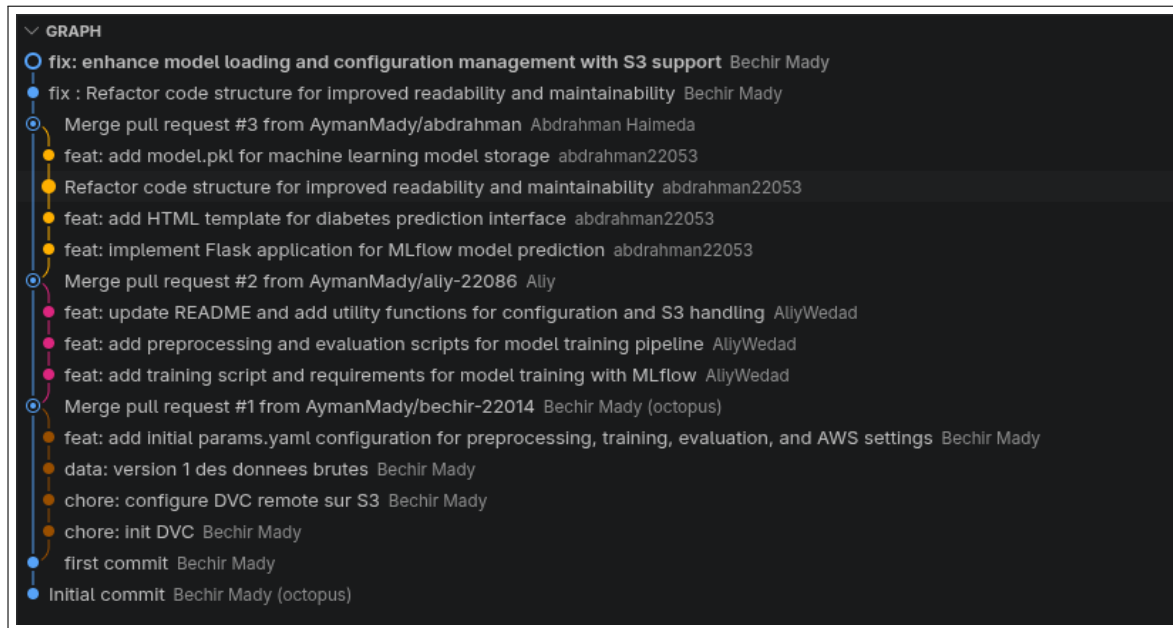


FIGURE 9 – Graphe Git (GitHub Network ou « `git log --graph` ») montrant les branches par membre et les merges.